

Neural CSSR: Fast Unsupervised Epsilon Machine Discovery Using Transformer-Based Probability Providers

Matteo Chinazzi
Neural CSSR Research Project

September 26, 2025

Abstract

We present Neural CSSR, a novel approach to computational mechanics that leverages neural networks as probability providers for Causal State Splitting Reconstruction (CSSR). Our key contribution is a fast unsupervised algorithm for epsilon machine discovery that combines transformer-based models with efficient three-stage clustering. The method achieves $O(k)$ complexity in Stage B through representative sampling and caching, while introducing backward stability refinement for minimal suffix identification. We demonstrate successful recovery of epsilon machines for processes including the Golden Mean, Even Process, and complex hierarchical systems, with validation through both ground truth comparison and epsilon machine loss evaluation.

1 Introduction

Computational mechanics provides a powerful framework for understanding the structure of stochastic processes through epsilon machines - minimal predictive models that capture the causal states of a system. Traditional CSSR algorithms rely on empirical frequency estimation, which becomes problematic for sparse data and complex dependencies.

Neural CSSR addresses these limitations by using neural networks, particularly transformer architectures, as sophisticated probability providers. Our approach enables:

- **Robust probability estimation** for rare contexts through neural generalization
- **Efficient unsupervised discovery** via multi-stage clustering with $O(k)$ complexity
- **Backward stability analysis** for minimal suffix identification
- **Scalable state remerging** to handle infinite memory processes

2 Background: Epsilon Machines and CSSR

2.1 Epsilon Machines

An epsilon machine $\mathcal{E}$ for a stochastic process is defined by:

- A set of causal states $\mathcal{S} = \{S_i\}$
- Transition probabilities $P(S_j|S_i, x)$ for symbol x
- Emission probabilities $P(x|S_i)$ from each state

The causal states partition histories by their predictive equivalence:

$$S_i = \{h : P(X_t|H_t = h) = P(X_t|S_i)\}$$

2.2 Classical CSSR Limitations

Traditional CSSR faces several challenges:

1. **Data sparsity:** Long contexts appear infrequently in finite data
2. **Computational complexity:** $O(n^2)$ pairwise comparisons
3. **Memory requirements:** Storing all observed contexts
4. **Threshold sensitivity:** JS divergence thresholds require careful tuning

3 Neural CSSR Framework

3.1 Neural Probability Providers

We employ transformer-based language models as probability providers, specifically:

Energy-Based Binary Language Models (EBM-BLM):

$$E(x|h) = \text{energy_head}(\text{transformer}(h)) \tag{1}$$

$$P(x|h) = \frac{\exp(-E(x|h))}{\sum_{x'} \exp(-E(x'|h))} \tag{2}$$

Autoregressive Binary Language Models (AR-BLM):

$$\text{logits} = \text{lm_head}(\text{transformer}(h)) \tag{3}$$

$$P(x|h) = \text{softmax}(\text{logits}) \tag{4}$$

Both architectures achieve comparable performance:

- **Golden Mean:** ~ 0.66 bits/symbol (theoretical optimum)
- **Even Process:** ~ 0.72 bits/symbol with proper parity detection

3.2 Model Training and Calibration

Our models are trained on binary sequences with:

- Context windows: 64-128 tokens
- Architecture: 4-8 transformer layers, 128 hidden dimensions
- Platt scaling for probability calibration

4 Fast Unsupervised Algorithm

4.1 Algorithm Overview

Our three-stage algorithm addresses the $O(n^2)$ bottleneck of traditional approaches:

Algorithm 1 Fast Unsupervised Epsilon Machine Discovery

- 1: **Input:** Histories H , model M , thresholds τ_A, τ_B
 - 2: **Stage A:** Distance-based emission clustering
 - 3: **for** each history $h \in H$ **do**
 - 4: Compute emission probabilities $P(\cdot|h)$
 - 5: **end for**
 - 6: Agglomerative clustering with JS divergence $< \tau_A$
 - 7: **Optional:** Backward stability refinement
 - 8: **Stage B:** Representative-based conditional JS clustering
 - 9: **for** each emission group **do**
 - 10: Select representatives (max k per group)
 - 11: Cluster representatives using conditional JS $< \tau_B$
 - 12: **end for**
 - 13: **Stage C:** State remerging for infinite memory processes
 - 14: **Output:** Discovered epsilon machine states
-

4.2 Stage A: Distance-Based Emission Clustering

Stage A groups histories with similar emission distributions:

1. Compute emission probabilities for all histories
2. Initialize singleton clusters
3. Repeatedly merge closest clusters (by centroid JS divergence)
4. Stop when minimum JS divergence exceeds threshold τ_A

This reduces the search space from $O(n^2)$ to $O(g^2)$ where $g \ll n$ is the number of emission groups.

4.3 Backward Stability Refinement

For each emission group, we identify minimal predictive suffixes:

Algorithm 2 Backward Stability Test

```

1: Input: History  $h$ , tolerance  $\epsilon$ 
2:  $P_{\text{full}} \leftarrow P(\cdot|h)$ 
3: for  $\ell = |h| - 1$  down to  $L_{\min}$  do
4:    $h_\ell \leftarrow h[-\ell :]$  {Suffix of length  $\ell$ }
5:    $P_\ell \leftarrow P(\cdot|h_\ell)$ 
6:    $js \leftarrow \text{JS}(P_{\text{full}}, P_\ell)$ 
7:   if  $js > \epsilon$  then
8:     return  $h_{-(\ell+1)}$  {Need one more token}
9:   end if
10: end for
11: return  $h_{-L_{\min}}$  {Minimal suffix is stable}

```

This finds the shortest suffix h^* such that $\text{JS}(P(\cdot|h), P(\cdot|h^*)) \leq \epsilon$.

4.4 Stage B: Efficient Representative Clustering

Stage B avoids $O(n^2)$ conditional JS computations through representative sampling:

1. Select up to k representative histories per emission group
2. Cluster representatives using conditional JS divergence
3. Utilize k-step distribution caching for efficiency
4. Assign remaining histories to closest representative clusters

The conditional JS divergence measures predictive similarity:

$$\text{JS}_{\text{cond}}(h_1, h_2) = \sum_x P(x|h_i) \text{JS}(P(\cdot|h_i x), P(\cdot|h_j x))$$

4.5 Stage C: State Remerging

For infinite memory processes, functionally identical states may be discovered as separate clusters. Stage C remerges states that satisfy both:

1. Emission similarity: $\text{JS}(P(\cdot|s_i), P(\cdot|s_j)) < \tau_{\text{em}}$
2. Rollout similarity: $\text{JS}_k(s_i, s_j) < \tau_{\text{roll}}$

Process	States	Entropy Rate	Description
Golden Mean	2	2/3 bits	Fibonacci substitution
Even Process	2	~ 0.72 bits	Parity-based transitions
Seven-State Human	7	Variable	Complex hierarchical

Table 1: Test Processes for Epsilon Machine Recovery

5 Experimental Results

5.1 Test Processes

We evaluate on canonical computational mechanics processes:

5.2 Recovery Performance

Golden Mean Recovery:

- Discovered: 2 states (matches ground truth)
- Purity: 100% state assignment accuracy
- Loss: 0.656 bits/symbol (near optimal)

Even Process Recovery:

- Discovered: 2 states with proper parity detection
- Purity: 96-98% with backward stability
- Context requirement: minimum 128 tokens for parity learning

Seven-State Human Recovery:

- Discovered: 7 states matching ground truth patterns
- Minimal suffixes: 'bb', 'aaa', 'aaab', 'ba', 'bab', 'baab', 'baa'
- Backward stability reduces contexts by average 2.3 tokens

5.3 Computational Efficiency

Our algorithm achieves significant speedups:

Stage	Traditional CSSR	Fast Algorithm
Emission Clustering	$O(n^2)$	$O(n \log n)$
Conditional JS	$O(n^2)$	$O(k^2)$ where $k \leq 10$
Cache Performance	N/A	85% hit rate

Table 2: Computational Complexity Comparison

5.4 Hyperparameter Sensitivity

Key findings for robust recovery:

- **Stage A threshold:** 0.001-0.025 (higher = fewer emission groups)
- **Stage B threshold:** 0.001-0.02 (higher = fewer final states)
- **Backward stability tolerance:** 1e-3 to 1e-2 bits
- **Representatives per group:** 5-10 (diminishing returns beyond 10)

6 Novel Contributions

6.1 Backward Stability Analysis

Our backward stability test identifies minimal predictive suffixes within each emission group. This addresses the "infinite memory problem" by finding the shortest context that preserves predictive power:

$$h^* = \arg \min_{|s|} |s| \text{ subject to } \text{JS}(P(\cdot|h), P(\cdot|s)) \leq \epsilon$$

where s is a suffix of history h .

6.2 Three-Stage Clustering Architecture

The combination of emission clustering, representative sampling, and state remerging provides a principled approach to scalable epsilon machine discovery:

1. **Stage A:** Reduces search space by grouping similar emissions
2. **Stage B:** Achieves $O(k)$ complexity through representatives
3. **Stage C:** Handles infinite memory through functional equivalence

6.3 Neural Probability Integration

Our framework seamlessly integrates neural probability providers with classical CSSR, enabling:

- Robust estimation for sparse contexts
- Learned representations capturing long-range dependencies
- Calibrated probabilities through Platt scaling

7 Validation Methodology

7.1 Ground Truth Comparison

We validate discovered states against known ground truth through:

- **Purity:** Fraction of histories in each cluster belonging to dominant ground truth state
- **Coverage:** Fraction of expected ground truth states discovered
- **State count accuracy:** Matching the expected number of causal states

7.2 Epsilon Machine Loss Evaluation

We compute the negative log-likelihood that the discovered epsilon machine achieves on held-out data:

$$L = -\frac{1}{N} \sum_{i=1}^N \log P(x_i | s_{i-1})$$

where s_{i-1} is the predicted state for context h_{i-1} .

7.3 Inductive Bias Probing

We evaluate model representations using Inductive Bias Probing (IBP):

- **R-IB:** Fraction of same-state pairs with identical predictions
- **D-IB:** 1 - fraction of different-state pairs with identical predictions
- Recent results: Even Process R-IB ≈ 0.98 , D-IB ≈ 0.96

8 Implementation Details

8.1 Caching and Optimization

Our implementation includes several optimizations:

Listing 1: K-Step Distribution Caching

```
class KStepCache:
    def __init__(self):
        self.cache = {}
        self.hits = 0
        self.misses = 0

    def get(self, model, hist, k, platt_params=None):
        key = f"{tuple(hist)}_{k}"
        if key in self.cache:
            self.hits += 1
            return self.cache[key]

        # Compute and cache
        dist = get_kstep_distribution(model, hist, k, platt_params)
        self.cache[key] = dist
        self.misses += 1
        return dist
```

8.2 Sampling Strategies

We implement two sampling strategies for history selection:

Random Sampling: Uniform selection from all possible contexts **Emission Stratified Sampling:** Stratified by emission probabilities to ensure diverse coverage

9 Future Directions

9.1 Theoretical Extensions

- **Convergence guarantees:** Theoretical analysis of algorithm convergence properties
- **Sample complexity:** Bounds on required data for accurate recovery
- **Approximation quality:** Relationship between neural approximation and true CSSR

9.2 Algorithmic Improvements

- **Adaptive thresholding:** Automatic threshold selection based on data characteristics
- **Hierarchical clustering:** Multi-resolution state discovery
- **Online learning:** Incremental epsilon machine updating

9.3 Application Domains

- **Natural language processing:** Syntactic state discovery in linguistic data
- **Biological sequences:** Protein and DNA motif identification
- **Time series analysis:** Hidden state discovery in temporal data

10 Conclusion

Neural CSSR represents a significant advancement in computational mechanics, combining the theoretical rigor of epsilon machines with the practical power of neural networks. Our fast unsupervised algorithm achieves:

- **Scalability:** $O(k)$ complexity in critical bottleneck stages
- **Robustness:** Successful recovery across diverse stochastic processes
- **Efficiency:** 85% cache hit rates and minimal suffix identification
- **Accuracy:** High purity and coverage for ground truth validation

The framework opens new possibilities for understanding complex stochastic systems through the lens of computational mechanics, with neural networks providing the probabilistic foundation for reliable epsilon machine discovery.

11 Code Availability

The complete implementation is available at:
<https://github.com/yourusername/NeuralCSSR>

Key files:

- `nanoGPT/js_analysis/unsupervised_fast_original.py`: Main algorithm
- `experiments/ebm/models.py`: Neural probability providers
- `run_neural_cssr.py`: Full pipeline integration

References

- [1] J.P. Crutchfield and B.S. McNamara. Equations of motion from a data series. *Complex Systems*, 1:417–452, 1987.
- [2] C.R. Shalizi and J.P. Crutchfield. Computational mechanics: Pattern and prediction, structure and simplicity. *Journal of Statistical Physics*, 104(3-4):817–879, 2001.

- [3] A. Vaswani, N. Shazeer, N. Parmar, et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- [4] J. Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. *Advances in Large Margin Classifiers*, 10(3):61–74, 1999.